

Lab session 2: Programming Fundamentals

In this lab session, you are only allowed to make use of constructs that are taught in the first two weeks of the course (so no arrays, functions, recursion, etc).

As explained in the lecture, to pass the tests in Themis the output of your program must be terminated by a newline character (`\n`).

Problem 1: Mulsum numbers

A positive integer is called a *mulsum number* if the product of its digits is equal to the sum of its digits. For example, 312 is a mulsum number because

$$3 \cdot 1 \cdot 2 = 3 + 1 + 2 = 6.$$

Write a C program that reads a positive integer from the input and outputs YES if this number is a mulsum number, and NO otherwise.

Example 1:

input:

312

output:

YES

Example 2:

input:

12151

output:

YES

Example 3:

input:

13211122

output:

NO

Problem 2: Takuzu number

An unsigned `int` is called a *takuzu number* if its bit pattern contains no occurrence of three equal consecutive bits. Leading zeros are ignored.

For example, 83 is a takuzu number because its bit pattern is

$$83 = (1010011)_2,$$

which contains neither 000 nor 111. The number 7 is not a takuzu number because its bit pattern is $(111)_2$.

Write a C program that reads an unsigned `int` and determines whether it is a takuzu number.

Example 1:

input:

83

output:

YES

Example 2:

input:

35

output:

NO

Example 3:

input:

292

output:

YES

Problem 3: Collision

Given an integer M (where $1 \leq M < 10000$) and two integers p and q (where $1 \leq p, q < M$), repeatedly update both values according to $p \leftarrow (2p + 1) \bmod M$ and $q \leftarrow (3q + 1) \bmod M$.

A *collision* occurs when $p = q$. Write a C program that determines whether a collision will eventually occur. If so, print the number of steps required to reach the first collision and the value of p and q at that time. If no collision will ever occur, print `No collision`.

The initial values correspond to step 0. Thus, if the initial values of p and q are equal, a collision occurs after 0 steps.

Example 1:

input:

13 3 7

output:

2: p=q=2

Example 2:

input:

42 7 13

output:

4: p=q=1

Example 3:

input:

36 7 2

output:

No collision

Problem 4: Prime gaps

Given an integer n (where $1 \leq n \leq 150$), determine the smallest pair of consecutive prime numbers p and q , with $p < q$, for which $q - p \geq n$.

Two primes are *consecutive* if there is no prime number strictly between them. The smallest pair is the pair with the smallest possible value of p .

Write a C program that prints the difference $q - p$, followed by p and q , in the format as given in the examples below.

Example 1:

input:

2

output:

2: 3 5

Example 2:

input:

3

output:

4: 7 11

Example 3:

input:

42

output:

44: 15683 15727

Problem 5: Cyclic multiple

A positive integer is called a *cyclic multiple* if all cyclic rotations of its digits have a common factor greater than 1.

For example, 105 is a cyclic multiple. Its cyclic rotations are 105, 051, and 510, and all three numbers are divisible by 3: $105 = 3 \cdot 35$, $51 = 3 \cdot 17$, $510 = 3 \cdot 170$.

Write a C program that reads a positive integer and determines whether it is a cyclic multiple.

Example 1:

input:

105

output:

YES

Example 2:

input:

679

output:

NO

Example 3:

input:

579

output:

YES